PatientTriage.ai

> **Intelligent Emergency Department (ED) co-pilot separating clinical urgency from operational flow. Preventing dangerous delays during hospital surges without replacing human clinical judgment.**

Video Demo

🏥 The Clinical Problem & Our Solution

During hospital surges, standard triage bottlenecks can lead to dangerous delays in care. Traditional systems often treat the Emergency Department as a single priority queue, overwhelmed by ambiguous symptoms and lacking dynamic operational scheduling.

PatientTriage.ai fixes this by introducing a Two-Agent system:

1. **Safety Agent**: Accurately risk-stratifies patients based on deterministic medical math.
2. **Flow Agent**: Maps symptoms to parallel operational tasks to maximize ED throughput.

This system ensures that high-risk patients are flagged instantly while parallelizing care for others, entirely sidestepping the hallucination risks of traditional black-box AI approaches in healthcare.

🏗️ Core Architecture

Our architecture is designed around a **Hybrid AI** model to ensure patient safety and maintain absolute clinical transparency:

- **NLP Normalization (LLM)**: We utilize OpenRouter (`openai/gpt-oss-120b`) strictly as a **fuzzy lexical normalizer**. It translates typos, patient shorthand, and colloquialisms into standardized clinical symptoms.
- **Safety Agent (Risk Stratification)**: All medical math is handled by a transparent, 100% deterministic rules engine. It evaluates age-adjusted vital sign thresholds (Pediatric vs. Geriatric). It actively optimizes for **safe under-triage avoidance**—if a patient lacks medical history (e.g., first-time visit) or presents ambiguous symptoms, the agent mathematically drops its confidence score and biases toward escalation.
- **Flow Agent (Resource Scheduling)**: Instead of a linear queue, it maps symptoms to parallel operational tasks (e.g., `ORDER_ECG`, `DRAW_TROPONIN`), accelerating time-to-treatment.

✨ Key Features

- **Clinical Accountability (HIPAA Override)**: Strict clinician override mechanism. Any manual change to the AI's triage score triggers a Prisma `$transaction` that securely logs the clinician's role, original score, and a mandatory justification into an immutable PostgreSQL audit trail.
- **Surge Simulation & Ongoing Monitoring**: A dynamic UI feature simulating a 3x volume surge. It actively monitors wait times in the queue, visually pulsing red/amber if patients breach safe time

thresholds for their severity level. Includes a 'Re-assess' workflow for deteriorating patients.
- **EHR Integration Simulation**: Includes an MRN (Medical Record Number) Lookup tool that allows returning patients to instantly auto-populate their existing medical histories.

💻 Tech Stack

- **Frontend**: React, TypeScript, Vite, Material-UI (MUI)
- **Backend**: Express.js, TypeScript, Prisma ORM
- **Database**: PostgreSQL (Containerized via Docker)
- **AI/NLP**: OpenRouter (`openai/gpt-oss-120b`) via the OpenAI SDK

⚖️ For Judges: Local Setup & Testing

We have designed this project to be easily runnable locally for evaluation. The application comes pre-loaded with a deterministic seed of 20 diverse edge-case patients (pediatric, geriatric, first-time, ambiguous) to demonstrate the system's robustness.

Prerequisites

- Node.js (v18+)
- Docker & Docker Compose

1. Environment Setup

Rename the example environment file and add your OpenRouter API key:

```
cd backend
cp .env.example .env
# Open .env and add your OPENROUTER_API_KEY
```

2. Start the Database

Spin up the containerized PostgreSQL database:

```
docker compose up -d
```

3. Database Reset & Seeding (Demo State)

To instantly reset the database to a clean demo state with the 20 pre-configured edge-case patients, run the following command from the `backend` directory:

```
npm run db:restart
```

(Note: This runs `prisma migrate reset --force && npm run db:seed` behind the scenes).

4. Start the Application

In separate terminal windows, start the backend and frontend development servers:

Backend:

```
cd backend
npm install
npm run dev
```

Frontend:

```
cd frontend
npm install
npm run dev
```
